

CT Praktikum: Modulare Programmierung – Linker

1 Einleitung

In diesem Praktikum lernen Sie, Fehler bei der Verwendung fremder Libraries zu beheben und wie Sie den Debugger dazu bringen, auch in die Library Funktionen hineinzuspringen. Zusätzlich lernen Sie die Input- und Output-Dateien beim Linken von Programmen mit statischen Libraries zu interpretieren.

Das Projekt besteht aus einer Library mit dazu passenden Header Files.

Sie müssen in der Entwicklungsumgebung die korrekten Speicherorte für Header Files angeben und fehlende Include-Direktiven in `main.c` einfügen. Weiterhin sollen Sie bei den Linker-Optionen richtig angeben, wo der Linker die Library findet.

Danach geht es darum, das Programm zu debuggen. Dabei lernen Sie, wie Sie beim Builden zwischen Libraries mit und ohne Debug-Information wechseln, und wie Sie dem Debugger sagen, wo er die benötigten Source-Files für das Source-Level-Debugging findet.

2 Lernziele

- Sie können Compiler- und Linker-Fehlermeldungen, welche im Zusammenhang mit modularer Programmierung auftreten können, interpretieren und korrigieren.
- Sie können Libraries inklusive notwendiger Header Files einbinden.
- Sie können ELF Symbol Sections ausgeben und interpretieren.
- Sie können Linker Map Files interpretieren.

3 Aufgaben

3.1 Aufgabe 1: Compiler und Linker Fehlermeldungen interpretieren

In einem ersten Schritt soll das unfertige Projekt zum Laufen gebracht werden. Dazu müssen die Fehlermeldungen des Präprozessors, des Compilers und des Linkers interpretiert und korrigiert werden.

Siehe dazu die Task-Liste in der Datei `main.c`.

Modulare Programmierung bedeutet für dieses Projekt, dass neben dem Code im `app` Ordner zusätzliche Library Header Files im `.inc` Ordner liegen. Dieser Ordner muss an geeigneter Stelle in den Projekt Properties (C/C++ Tab) angegeben werden, damit diese Header Dateien vom Präprozessor/Compiler auch gefunden werden.

Analog müssen die einzelnen Libraries im Linker Tab der Projekt Properties unter *Misc Controls* dem `lib` Ordner angegeben werden. Geben sie dazu die benötigte Library in diesem Feld ein (z.B. `lib\read_write.lib`)

Wenn Sie alle Fehler erfolgreich korrigiert haben, können sie das Programm auf das CT-Board laden.

Die Funktion des Programms ist simpel: Wenn Sie auf T0 drücken, wird beim ersten Mal drücken von dunkel auf ein fixes Muster gewechselt, mit jedem weiteren T0 Drücken, werden die LEDs invertiert.

3.2 Aufgabe 2: Debugging

- a) Versuchen Sie das Programm im Debugger in Einzelschritten auszuführen (**Step/F11**). Was beobachten Sie bei der Funktion `read8(BUTTONS)`? (Infos zu Debugging finden Sie auf CT Board Wiki (<https://ennis.zhaw.ch>) unter «Compile and Debugging»)

Was beobachten Sie?

- b) Ersetzen Sie im Linker Tab die Referenz auf `lib\read_write.lib` mit `lib_debug\read_write.lib`. Was beobachten Sie, wenn Sie nun kompilieren, linken und debuggen?
- c) Schliesslich ersetzen sie die Referenz durch `lib_debug_with_src\read_write.lib`. Beim Debugging mit Sourcen müssen Sie dem Debugger zusätzlich noch angeben, wo sich die Source-Files genau befinden. Im Library File selbst befinden sich nämlich keine Source-Files, lediglich die Symbole. Geben sie hierfür im *Command Window* des Debuggers folgenden Befehl ein:

```
set src = C:\<Ihr Pfad>\project\lib_debug_with_src
```

Der Debugger weiss jetzt, wo sich die Source-Files befinden. Achten Sie darauf, dass es im Pfad kein Leerzeichen hat.

Was beobachten Sie, wenn Sie nun kompilieren, linken und debuggen?

Was ist der Grund für das veränderte Verhalten?

3.3 Aufgabe 3: Symbole extrahieren

Das Tool `fromelf.exe` kann den Inhalt der binären ELF-Dateien in lesbarer Form ausgeben. Die Objekt Dateien (file.o), die Libraries (file.lib) und die Programme (file.axf) sind alle in ELF File Format gegeben.

Führen Sie `fromelf.exe` in einem Command Prompt aus.

Ein Command Prompt öffnen Sie, indem sie `cmd.exe` ausführen.

Das Tool `fromelf.exe` wird über diesen Pfad ausgeführt (Pfad kann je nach Installationsverzeichnis der KEIL Toolchain abweichen):

```
C:\Keil_v5\ARM\ARMCLANG\bin\fromelf.exe.
```

z.B.

```
Command Prompt
C:\Temp>C:\KEIL_v5\ARM\ARMCLANG\bin\fromelf.exe
Product: MDK-ARM Lite 5.40
Component: Arm Compiler for Embedded 6.22
Tool: fromelf [5ee90f00]

ARM image conversion utility
fromelf [options] input_file

Options:
--help          display this help screen
--vsn           display version information
--output file   the output file. (defaults to stdout for -text format)
--nodebug       do not put debug areas in the output image
--nolinkview    do not put sections in the output image

Binary Output Formats:
--bin           Plain Binary
--m32           Motorola 32 bit Hex
--i32           Intel 32 bit Hex
--vhx          Byte Oriented Hex format

--base addr    Optionally set base address for m32,i32

Output Formats Requiring Debug Information
--fieldoffsets Assembly Language Description of Structures/Classes
--expandarrays Arrays inside and outside structures are expanded

Other Output Formats:
--elf          ELF
--text         Text Information

Flags for Text Information
-v            verbose
-a            print data addresses (For images built with debug)
-c            disassemble code
-d            print contents of data section
-e            print exception tables
-g            print debug tables
-r            print relocation information
-s            print symbol table
-t            print string table
-y            print dynamic segment contents
-z            print code and data size information
```

Generieren Sie für *Objects\toggle.o*, *Objects\main.o* und *lib\read_write.lib* die Symbol Tabelle. Fokussieren Sie auf Code und Data Einträge und ignorieren Sie Debug Einträge

Welches sind lokale Symbole?

Welches sind exportierte Symbole?

Welches sind importierte Symbole (referenzierte Symbole)?

Hinweis: Gesucht sind nicht die Namen der einzelnen Symbole, sondern mit welcher Bezeichnung diese in der generierten Tabelle hinten markiert werden.

Lokal	Importiert	Exportiert

Vergleichen Sie die obige Antwort mit dem entsprechenden Header File.

Was ist mit den als exportierten Symbolen gemeldeten Einträgen, die nicht im Header File stehen?

Wenn Sie eine Library haben und keine dazu passende Header Files, können Sie dann mit dem *fromelf.exe* Tool alle nötigen Informationen aus der Library extrahieren, um selbst ein Header File zu schreiben? Fehlt etwas?

3.4 Aufgabe 4: Linker Map Interpretieren

Beim Linken wird ein Map File generiert. Prüfen Sie, welche der Informationen unter "Project" → "Options for Target ..." → Tab "Listing" im Map File vorkommen.

Erklären Sie anhand des Linker Map Files, wie das Memory Map aussieht.

Suchen Sie zunächst die «Memory Map» Section im Linker Map File.

Wo sind welche Konstanten, welche Funktionen und welche Daten abgelegt?

3.5 Bewertung

Die lauffähigen Programme müssen präsentiert werden. Die einzelnen Studierenden müssen die Lösungen und den Quellcode verstanden haben und erklären können.

Aufgabe	Ziel	Gewichtung
3.1	Fehler und Warnungen behoben, Programm läuft wieder.	1/4
3.2	Kann erklären wieso der Debugger gewisse Funktionen "überspringt".	1/4
3.3	Symbole extrahiert, Tabelle korrekt ausgefüllt.	1/4
3.4	Interpretation der Linker Map korrekt, weiss wo man Konstanten, Funktionen und Daten findet.	1/4